

SUPPLEMENTARY · MODULE 3

Spring Boot Microservices Development

Stand up production-shaped Spring Boot services: starters, profiles, actuator, externalized configuration, and the packaging habits that survive OpenShift.



INTRODUCTION TO THE SPRING ECOSYSTEM · SPRING FRAMEWORK VS. SPRING BOOT

✦ TRADITIONAL SPRING VS. SPRING BOOT

TRADITIONAL SPRING	SPRING BOOT
More manual configuration	Convention-based setup
External application server often required	Embedded server included
Dependencies configured individually	Starter dependencies
Slower project setup	Faster project creation
More boilerplate	Less boilerplate



THE SPRING FAMILY

- **Spring Framework:** the core container, dependency injection, and AOP foundation
- **Spring Boot:** convention-based setup, auto configuration, embedded servers
- Spring Data, Spring Security, Spring Cloud, Spring for Apache Kafka -- specialized modules built on the same core



WHY IT FITS BANKING MICROSERVICES

- Customer, Account, Transaction, Loan, and Notification services can all be built with Spring Boot
- Each service can be developed, tested, packaged, and deployed independently

KEY TAKEAWAY Spring Framework, Spring Boot, Spring Cloud, Spring Data, and Spring Security together cover nearly every microservice concern. • Traditional Spring required extensive manual configuration; Spring Boot replaces it with sensible defaults and starter dependencies.

SPRING BOOT ARCHITECTURE

Controller, service, and repository layers each have one job -- the discipline that keeps a Spring Boot microservice maintainable.



THE LAYERED FLOW

- Client -> REST Controller -> Service Layer -> Repository Layer -> Database
- **Controller:** maps URLs, reads input, calls the service, returns status codes
- **Service:** applies business rules, coordinates workflows, calls repositories
- **Repository:** reads, saves, updates, and deletes data

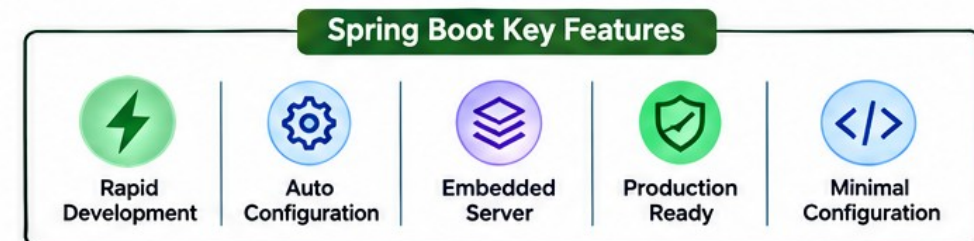
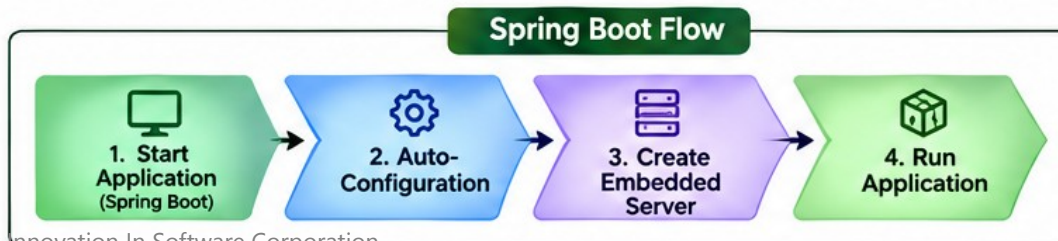
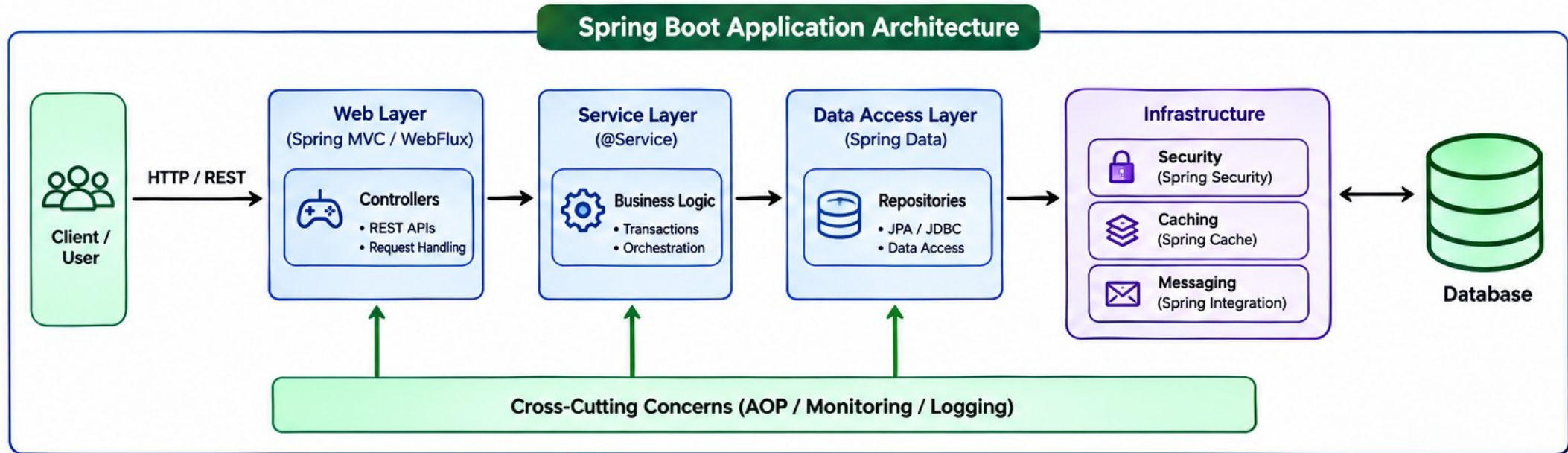
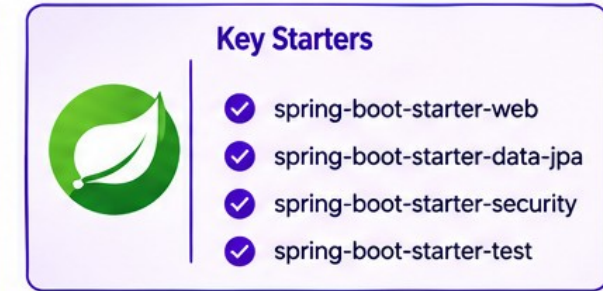
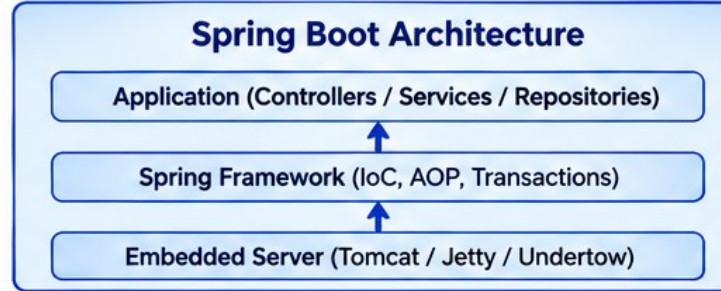
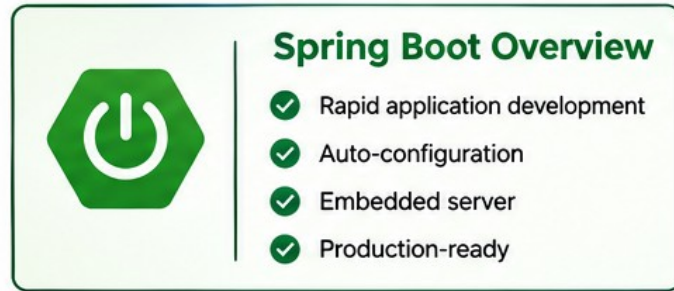


COMMON MISTAKES

- Putting business logic directly in controllers
- Returning database entities directly from APIs
- Mixing data access code with API code

KEY TAKEAWAY Controller, service, and repository layers each have one job -- the discipline that keeps a Spring Boot microservice maintainable.

Spring Boot Overview and Architecture



SPRING BOOT APPLICATION LIFECYCLE · SPRING BOOT PROJECT STRUCTURE



APPLICATION STARTUP FLOW

1. Create the Spring application context
2. Read configuration properties
3. Scan packages for components
4. Create and wire beans
5. Apply auto configuration
6. Start the embedded web server and expose REST endpoints



COMMON STARTUP PROBLEMS

- Missing dependency, invalid property value, bean not found
- Port already in use, database connection failure, package scanning issue



BANKING EXAMPLE: CUSTOMER SERVICE STRUCTURE

- com.bank.customer.controller -> CustomerController
- com.bank.customer.service -> CustomerService
- com.bank.customer.repository -> CustomerRepository
- com.bank.customer.model -> Customer
- com.bank.customer.dto -> CustomerRequest, CustomerResponse
- com.bank.customer.exception -> CustomerNotFoundException

KEY TAKEAWAY Context creation, configuration loading, component scanning, bean wiring, auto configuration, and server startup happen in a defined order. • Controller, service, repository, model, dto, and exception packages give every service in the codebase the same recognizable shape.

Spring Initializr and Project Structure

Spring Initializr



Spring Initializr is a web tool that helps you generate Spring Boot projects with the dependencies you need.

1. Project

- ☒ Maven Project
☐ Gradle Project

2. Language

- ☒ Java ☐ Kotlin ☐ Groovy

3. Spring Boot

3.2.5

4. Packaging

- ☒ Jar ☐ War

5. Java

17

6. Dependencies

Search dependencies...

Spring Web X Spring Data JPA X H2 Database X
Spring Boot DevTools X

GENERATE Ctrl + ↵

EXPLORE Ctrl + Space

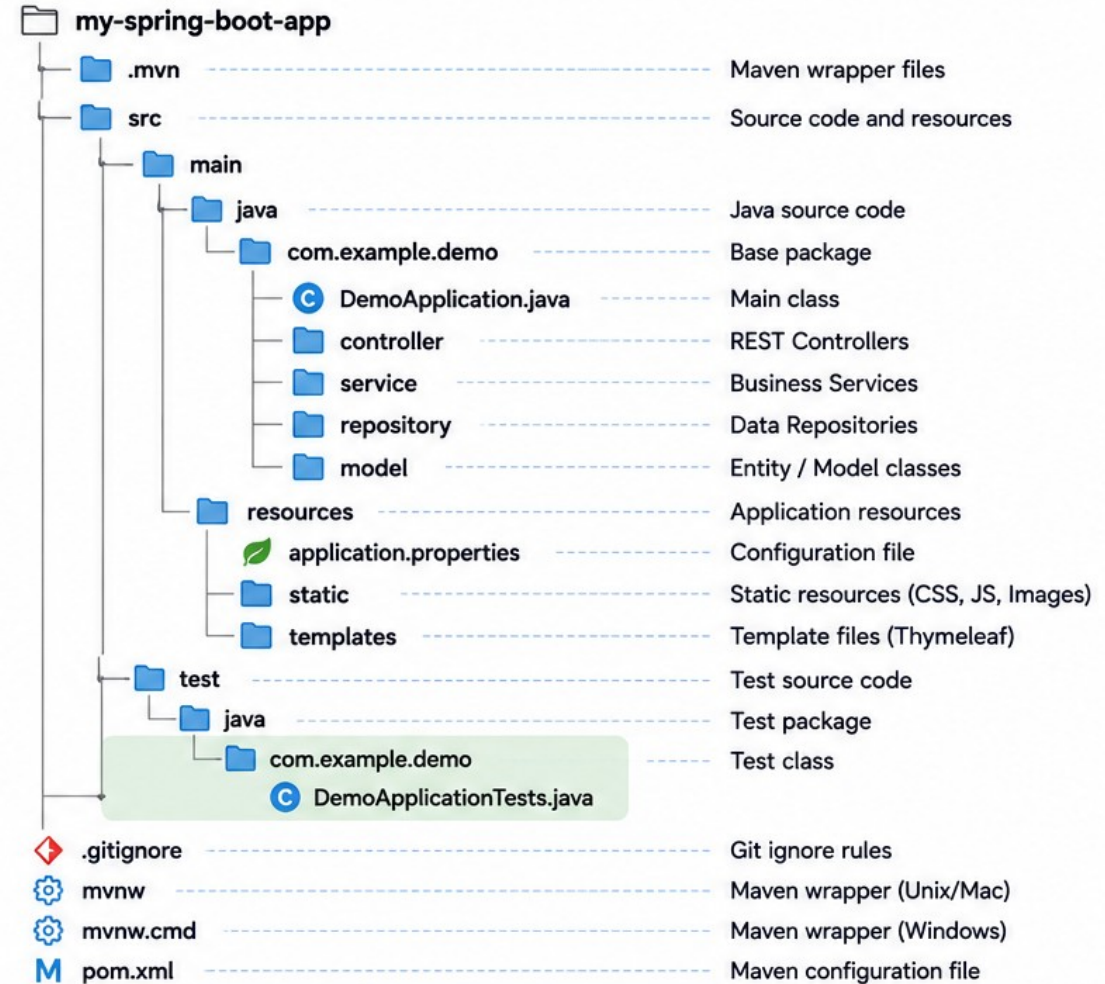
SHARE...



Download the generated project and import it into your IDE.

Project Structure

A typical Spring Boot (Maven) project structure:



Folder Java Class Properties File Maven File Config File Executable

CREATING PROJECTS WITH SPRING INITIALIZR · MAVEN AND GRADLE DEPENDENCY MANAGEMENT



COMMON PROJECT SETTINGS

- **Project:** Maven Language: Java
Packaging: JAR Java version: 17 or 21
- **Group:** com.bank Artifact: customer-service



COMMON DEPENDENCIES FOR A BASIC MICROSERVICE

- Spring Web, Validation, Spring Data JPA, PostgreSQL Driver or H2, Actuator
- Choose only the dependencies needed for the service



WHY BUILD-TOOL DEPENDENCY MANAGEMENT MATTERS

- **Microservices depend on many libraries:** web, validation, persistence, logging, testing
- A parent POM or dependency management section aligns compatible versions automatically
- Reduces incompatible versions, duplicate dependencies, and runtime class loading errors

KEY TAKEAWAY Choosing Maven, Java, the right group/artifact, and only the dependencies a service needs avoids manual setup errors. • A build tool with dependency management prevents version conflicts and lets the whole team build the same artifact reliably.

Spring Boot Starter Dependencies



Spring Boot Starters are a set of convenient dependency descriptors that simplify your build configuration. They bundle commonly used libraries and auto-configuration settings for a specific purpose.

WHAT IS A STARTER?



A starter is a dependency that brings in a set of related dependencies and enables auto-configuration for a specific feature.

HOW IT WORKS



Add Starter
to your build file



Maven/Gradle
downloads required
dependencies



Auto-Configuration
configures Spring and
related components



Ready to Use
Feature is ready
with minimal setup

EXAMPLE IN pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```



Just this one dependency provides web development capabilities out of the box!

WHY USE STARTERS?

- ✓ Simplifies dependency management with a single entry.
- ✓ Reduces version conflicts.
- ✓ Enables Spring Boot auto-configuration.
- ✓ Speeds up development.
- ✓ Promotes best practices and recommended libraries.

EXAMPLE: spring-boot-starter-web

Adds Spring MVC, embedded Tomcat, Jackson, Validation, and more.

Transitive Dependencies Included (Partial List)

Library	Purpose
spring-web	Web and REST support
spring-webmvc	Spring MVC framework
spring-boot-starter-json (Jackson)	JSON serialization/deserialization
hibernate-validator	Bean validation (Jakarta Validation)
spring-boot-starter-tomcat	Embedded Tomcat server
spring-boot-starter-logging	Logging with Logback, SLF4J
spring-core, spring-context	Core Spring Framework
jakarta.annotation-api	Annotations API



When you add one starter, all the necessary dependencies (and their dependencies) are added automatically.

COMMON STARTERS AND THEIR PURPOSES

	spring-boot-starter-web	Build web, RESTful applications using Spring MVC and Tomcat.
	spring-boot-starter-data-jpa	Build JPA based applications with Spring Data and Hibernate.
	spring-boot-starter-security	Add Spring Security for authentication and authorization.
	spring-boot-starter-data-redis	Access Redis key-value store with Spring Data Redis.
	spring-boot-starter-amqp	Integrate with RabbitMQ using Spring AMQP.
	spring-boot-starter-actuator	Add production-ready features like health, metrics, monitoring.
	spring-boot-starter-test	Support for testing with JUnit, Mockito, AssertJ, and more.

NAMING CONVENTION

spring-boot-starter-**<feature>**

Example: **spring-boot-starter-web**



KEY BENEFITS



Faster
development



Less
configuration



Fewer version
conflicts



Auto-configuration
out of the box



Built with
best practices

BEST PRACTICES

- Use only the starters you need.
- Prefer Spring Boot starters over adding dependencies manually.
- Regularly review and update starter versions.



DEPENDENCY VERSION MANAGEMENT · AUTO-CONFIGURATION EXPLAINED



BEST PRACTICES

- Use Spring Boot starters when available
- Avoid specifying versions for managed dependencies unless necessary
- Keep dependencies minimal and review transitive dependencies
- Use the same Spring Boot version strategy across related services



HOW AUTO CONFIGURATION WORKS

- Spring Boot inspects classpath dependencies, existing beans, properties, and conditional annotations
- Spring Web present -> configure web support -> start embedded server -> enable REST controllers
- Defaults can always be overridden with explicit properties or beans

KEY TAKEAWAY Letting Spring Boot's managed versions do the work avoids incompatible libraries, duplicate dependencies, and security patch drift. • Adding Spring Web alone configures embedded Tomcat, Spring MVC, JSON serialization, and default error handling automatically.

Auto-Configuration and Starters

Auto-Configuration



Spring Boot auto-configuration automatically configures your application based on the dependencies on the classpath and your defined properties.

How Auto-Configuration Works



1. Classpath Scanning

Spring Boot scans the classpath for available libraries and components.



2. Condition Evaluation

Conditions defined in auto-configuration classes are evaluated.



3. Bean Creation

If conditions match, Spring Boot creates and configures beans.



4. Property Binding

Properties from application.properties or application.yml are bound to beans.



5. Application Ready

The application is ready to run with minimal manual configuration.

Auto-Configuration Sources



@EnableAutoConfiguration
Enables auto-configuration in Spring Boot applications.



META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
Lists all auto-configuration classes available to Spring Boot.

Example: DataSource Auto-Configuration



H2 Dependency Added

spring-boot-starter-data-jpa (includes H2)



Spring Boot Detects

Finds H2, DataSource, JPA on classpath



Auto-Configuration Triggers

DataSource, EntityManager, TransactionManager beans are created



Ready to Use

No XML or manual configuration required!



Key Benefit: Auto-configuration reduces boilerplate configuration and lets you focus on business logic.

Starters



Starters are a set of convenient dependency descriptors that you can include in your application. They bring in all the transitive dependencies you need for a particular feature.

How Starters Help



Simplify Dependencies

Bundle related dependencies together.



Consistent Versions

Ensure compatible versions of libraries.



Faster Development

Add powerful features with a single dependency.



Best Practices

Include recommended configurations and defaults.

Common Spring Boot Starters

Starter	Includes	Use Case
 spring-boot-starter-web	Spring MVC, Tomcat, Jackson, Validation, etc.	Build web applications and REST APIs
 spring-boot-starter-data-jpa	Spring Data JPA, Hibernate, DataSource, etc.	Use JPA with relational databases
 spring-boot-starter-security	Spring Security	Add authentication and authorization
 spring-boot-starter-data-mongodb	Spring Data MongoDB, MongoDB Driver	Use MongoDB with Spring Data
 spring-boot-starter-mail	Spring Mail	Send emails easily
 spring-boot-starter-test	JUnit, Mockito, AssertJ, Spring Test, etc.	Testing support for your application



Example: Adding **spring-boot-starter-web** automatically configures Spring MVC, an embedded server (Tomcat), JSON support (Jackson), and more!

CONDITIONAL CONFIGURATION · COMPONENT SCANNING · SPRING BEANS AND THE APPLICATION CONTEXT



COMMON CONDITIONAL ANNOTATIONS

- **@ConditionalOnClass:** create a bean only if a given class is on the classpath
- **@ConditionalOnProperty:** create a bean only if a property is set to a specific value
- **@ConditionalOnMissingBean:** create a default bean only if the developer hasn't defined one



WHAT GETS SCANNED

- Spring Boot scans the package of the main application class and its subpackages
- Classes annotated @RestController, @Service, @Repository, @Component, @Configuration are discovered automatically
- A class placed outside the scanned package structure will not be found



WHAT IS A BEAN

- A bean is an object managed by the Spring container
- **Examples:** controller objects, service objects, repository objects, configuration objects
- Spring creates these objects and injects dependencies where needed

KEY TAKEAWAY @ConditionalOnClass, @ConditionalOnProperty, and @ConditionalOnMissingBean are how auto configuration makes its decisions. • Spring Boot scans the main application class's package and subpackages for stereotype-annotated classes -- outside that, it finds nothing. • Controllers, services, repositories, and configuration objects are all beans -- objects whose lifecycle Spring owns, not your code.

INVERSION OF CONTROL · DEPENDENCY INJECTION PATTERNS



WHAT INVERSION OF CONTROL MEANS

- **Traditional code:** a class creates its own dependencies with new
- **IoC:** the framework creates objects and controls their lifecycle and wiring
- Dependency injection is the mechanism Spring uses to implement IoC



INJECTION STYLES

STYLE	RECOMMENDATION
Constructor injection	Preferred
Setter injection	Useful for optional dependencies
Field injection	Avoid for most production code

KEY TAKEAWAY Instead of a class deciding how to construct its own dependencies, the Spring container decides and hands them in. • Constructor injection is preferred in Spring Boot; setter injection suits optional dependencies; field injection should be avoided in production code.

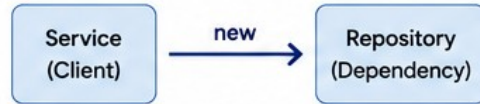
Dependency Injection



Dependency Injection (DI) is a design pattern in which an object receives its dependencies from an external source rather than creating them itself.

Without Dependency Injection

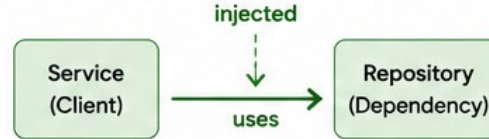
The class is responsible for creating its own dependencies.



❌ Tight coupling
Hard to test and maintain

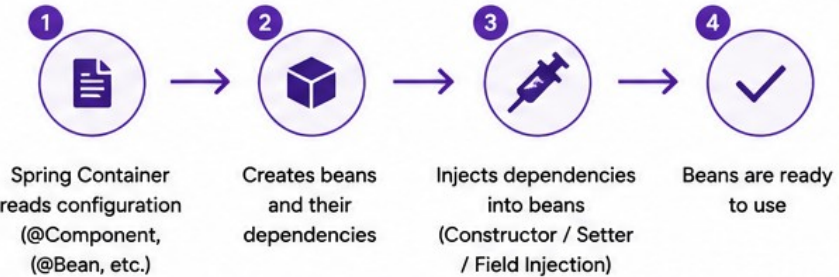
With Dependency Injection

Dependencies are provided from the outside.



✅ Loose coupling
Easy to test, maintain, and extend

How Dependency Injection Works in Spring



Example (Constructor Injection)

Service Class

```
@Service
public class UserService {
    private final UserRepository userRepository;

    // Dependency is injected
    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User getUser(Long id) {
        return userRepository.findById(id);
    }
}
```

Repository Interface

```
@Repository
public interface UserRepository {
    Optional<User> findById(Long id);
}
```

Spring Container creates UserRepository bean and injects it into UserService constructor.

Benefits of Dependency Injection



Loose Coupling

Classes depend on abstractions, not concretions.



Testability

Easier to unit test using mocks or stubs.



Maintainability

Changes in implementation don't affect clients.



Reusability

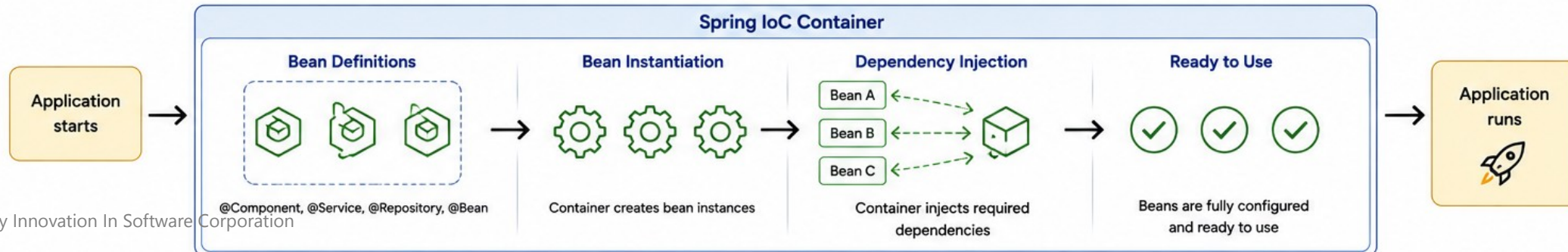
Components can be reused in different contexts.



Configurability

Dependencies can be changed without changing client code.

Spring IoC Container



CONSTRUCTOR INJECTION BEST PRACTICES

Final fields set through the constructor mean a service literally cannot exist in an incomplete or partially-wired state.



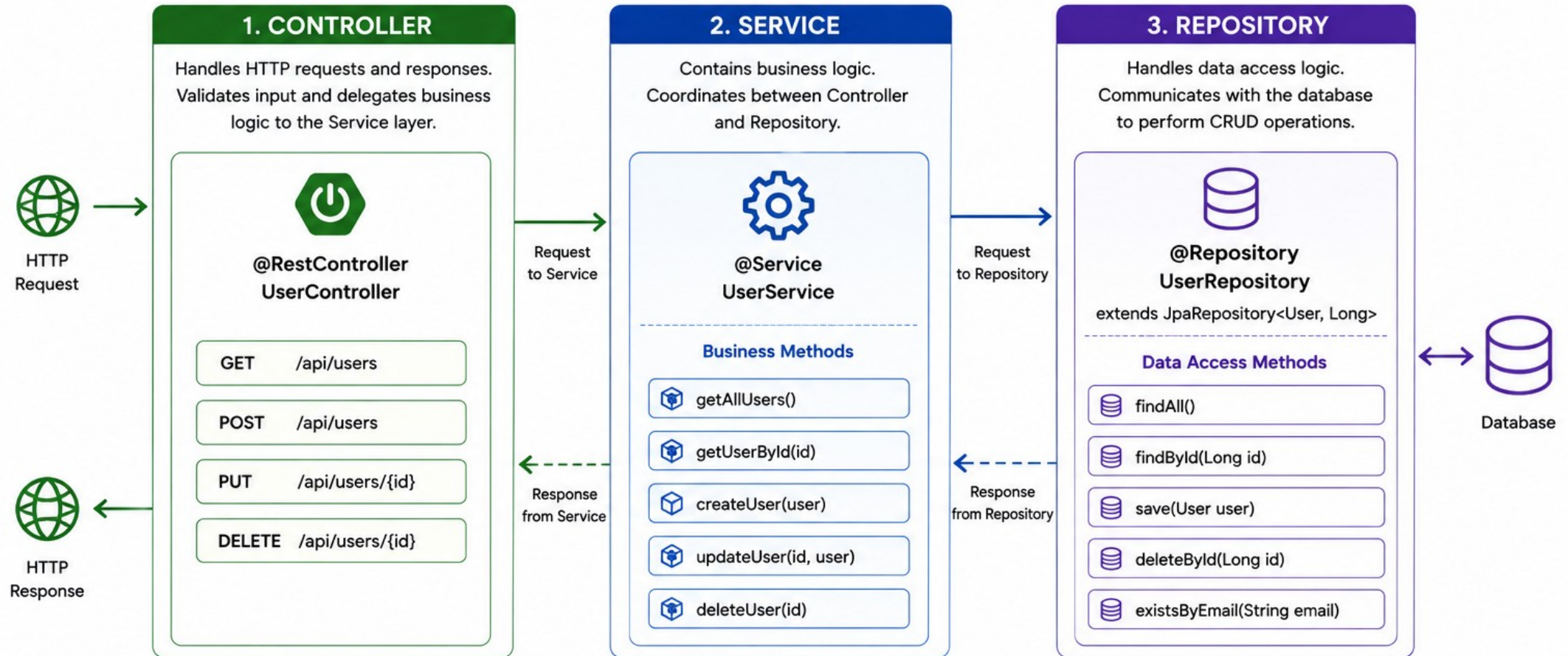
BENEFITS OF CONSTRUCTOR INJECTION

- Dependencies are explicit in the class's public signature
- Required dependencies cannot be missing at runtime
- Fields can be declared final, and unit testing is easier with plain constructor calls

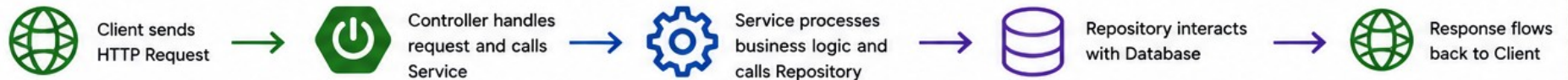
KEY TAKEAWAY Final fields set through the constructor mean a service literally cannot exist in an incomplete or partially-wired state.

Controllers, Services, Repositories

A layered architecture pattern in Spring Boot applications.
Each layer has a specific responsibility and communicates with the next layer.



FLOW SUMMARY



BEAN SCOPES AND LIFECYCLE · STEREOTYPE ANNOTATIONS · JAVA-BASED CONFIGURATION



COMMON BEAN SCOPES

- **Singleton (default):** one shared instance for the entire application context
- **Prototype:** a new instance every time the bean is requested
- **Request / Session:** scoped to a single HTTP request or session in web applications



COMMON STEREOTYPE ANNOTATIONS

- **@RestController:** marks a class as a REST API controller
- **@Service:** marks a class as containing business logic
- **@Repository:** marks a class as a data access component
- **@Component:** a generic Spring-managed bean; @Configuration marks configuration classes



WHEN TO USE JAVA CONFIGURATION

- Defining beans for third-party classes you cannot annotate directly
- Customizing a bean's construction beyond what auto configuration provides
- Grouping related bean definitions in one clear, type-checked class

KEY TAKEAWAY Singleton is the default and right choice for most services; request and prototype scopes exist for narrower, stateful use cases.

- @RestController, @Service, @Repository, and @Component all mark a class as a Spring-managed bean with a specific architectural role.
- @Configuration classes with @Bean methods give explicit control over bean creation when auto configuration and stereotypes aren't enough.

EXTERNALIZED CONFIGURATION · CONFIGURATION PROPERTIES



WHY EXTERNALIZE CONFIGURATION

- Values that change between environments should never be hardcoded in Java classes
- Externalized configuration allows the same application package to run in different environments
- Sources include application.yml, profile files, environment variables, and command-line arguments

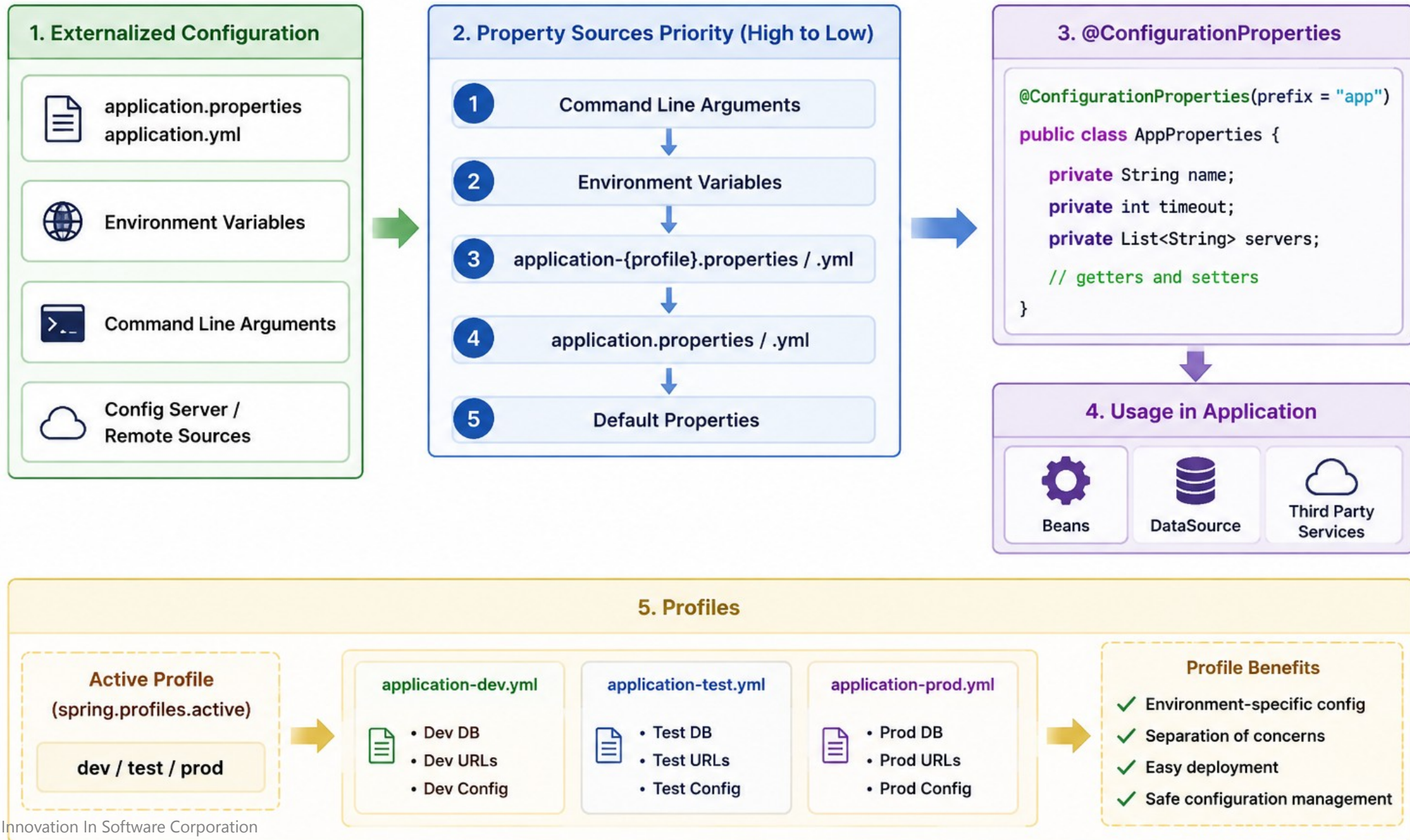


@VALUE VS. @CONFIGURATIONPROPERTIES

- @Value works for a single simple value, but becomes hard to manage as configuration grows
- @ConfigurationProperties(prefix = "customer") binds a whole group of related settings
- Grouped, type-safe configuration is easier to validate, test, and maintain

KEY TAKEAWAY Server port, database URL, feature flags, and timeouts should live outside the compiled code, in properties, YAML, or environment variables. • Instead of scattering @Value annotations, group related configuration into a single, prefixed, strongly typed properties class.

Configuration Properties and Profiles



PROFILES AND ENVIRONMENT MANAGEMENT · CONFIGURATION PRECEDENCE · ENVIRONMENT VARIABLES AND RUNTIME CONFIGURATION



ACTIVATING A PROFILE

- `spring.profiles.active=dev` in a properties file
- `java -jar customer-service.jar --spring.profiles.active=prod` from the command line



CONFIGURATION SOURCE PRECEDENCE (HIGH TO LOW)

1. Command-line arguments
2. Environment variables
3. Profile-specific configuration files
4. Base `application.yml` / `application.properties`
5. Default values defined in code



BEST PRACTICES

- Keep common settings in the base configuration file
- Keep production secrets out of source control
- Avoid too many profiles with unclear purpose



ENVIRONMENT VARIABLES IN CONTAINERS

- `SPRING_PROFILES_ACTIVE=prod`, `SERVER_PORT=8080`
- Spring Boot maps environment variables to configuration properties automatically
- Secrets (passwords, API keys) should be injected this way, never committed to source control

KEY TAKEAWAY `application-local.yml`, `application-dev.yml`, and `application-prod.yml` override the shared base configuration per environment. • Command-line arguments and environment variables override profile-specific files, which override the shared base `application.yml`. • On OpenShift, `SPRING_PROFILES_ACTIVE` and `SERVER_PORT` map directly onto Spring Boot configuration properties without a rebuild.

APPLICATION STARTUP AND SHUTDOWN · GRACEFUL SHUTDOWN

· SPRING BOOT DEVELOPER TOOLS



SHUTDOWN RESOURCES TO RELEASE

- Database connections and connection pools
- Thread pools and background workers
- Message consumers, file handles, and network connections



WHY GRACEFUL SHUTDOWN MATTERS

- `server.shutdown=graceful` lets Spring Boot stop accepting new requests while finishing active ones
- Essential for zero-downtime deployments on Kubernetes and OpenShift
- Pairs with readiness probes so the platform stops routing traffic before shutdown begins



WHAT DEVTOOLS PROVIDES

- Automatic application restart when classpath files change
- LiveReload browser refresh support
- Property defaults tuned for local development, not production
- Should be excluded from the production build

KEY TAKEAWAY Clean startup makes failures diagnosable; clean shutdown releases database connections, thread pools, and message consumers properly. • Without graceful shutdown, a rolling deployment or scale-down can abruptly cut off requests that were midway through processing. • Automatic restart on code changes, LiveReload, and sensible development-time defaults cut the friction out of local iteration.

PACKAGING EXECUTABLE APPLICATIONS · RUNNING SPRING BOOT APPLICATIONS · SPRING MVC REQUEST PROCESSING



WHAT'S INSIDE THE PACKAGE

- Compiled application classes and dependency libraries
- Resource files and Spring Boot launcher metadata
- Embedded server support -- no external application server required



THREE WAYS TO RUN LOCALLY

- **From the IDE:** run the main `@SpringBootApplication` class directly
- **With Maven:** `mvn spring-boot:run`
- **As a packaged JAR:** `java -jar target/customer-service-1.0.0.jar`



THE REQUEST PROCESSING FLOW

1. `DispatcherServlet` receives the incoming HTTP request
2. Handler mapping resolves the matching controller method
3. The controller method executes and returns a result
4. A message converter (Jackson) serializes the result to JSON
5. The HTTP response is returned to the client

KEY TAKEAWAY `mvn clean package` produces one JAR containing the compiled code, dependencies, and embedded server -- ready to run anywhere Java runs. • IDE runs suit day-to-day development; `mvn spring-boot:run` suits quick checks; the packaged JAR is closest to how the service runs in production. • `DispatcherServlet` receives the request, resolves the matching controller method, and converts the return value into an HTTP response.

BUILDING REST CONTROLLERS · REQUEST MAPPING AND HTTP METHODS



CUSTOMER API EXAMPLE

- `@RestController`
`@RequestMapping("/customers")`
- `@GetMapping("/{id}")`
`getCustomer(@PathVariable Long id)`
- `@PostMapping`
`createCustomer(@RequestBody CustomerRequest request)`



BEST PRACTICES

- Keep controllers thin -- delegate business logic to services
- Use DTOs for request and response payloads
- Return appropriate HTTP status codes



MAPPING ANNOTATIONS

ANNOTATION	HTTP METHOD	PURPOSE
@GetMapping	GET	Retrieve data
@PostMapping	POST	Create data
@PutMapping	PUT	Replace or update data
@PatchMapping	PATCH	Partially update data
@DeleteMapping	DELETE	Delete data

KEY TAKEAWAY `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, and `@DeleteMapping` map cleanly onto the five core REST operations. • `@RequestMapping` at the class level combines with method-level `@GetMapping`/`@PostMapping` to build a clean, resource-oriented API surface.

PATH VARIABLES AND QUERY PARAMETERS · REQUEST HEADERS AND COOKIES · REQUEST AND RESPONSE BODIES



PATH VARIABLES

- @GetMapping("/{id}") getCustomer(@PathVariable Long id)
- **Request:** GET /customers/101



QUERY PARAMETERS

- @RequestParam reads optional values from the query string
- **Example:** GET /customers?status=ACTIVE&page=0&size=20



READING HEADERS IN A CONTROLLER

- @RequestHeader("Authorization") String token
- @RequestHeader("X-Correlation-Id") String correlationId
- @CookieValue for reading cookie values when needed



REQUEST AND RESPONSE FLOW

- @PostMapping createCustomer(@RequestBody CustomerRequest request)
- **Incoming JSON:** {"firstName": "Asha", "lastName": "Patel", "email": "asha.patel@example.com"}
- The method's return value is serialized back to JSON automatically

KEY TAKEAWAY @PathVariable pulls an identifier out of the URL path; @RequestParam reads optional filters, paging, and sorting from the query string. • Authorization tokens, correlation IDs, and content negotiation all travel through headers -- accessed in Spring with @RequestHeader. • Jackson automatically converts an incoming JSON payload into a Java object, and a returned Java object back into a JSON response.

JSON SERIALIZATION WITH JACKSON · DATA TRANSFER OBJECTS · DOMAIN MODELS VS. API MODELS



COMMON JACKSON ANNOTATIONS

- **@JsonProperty:** rename a field in the JSON representation
- **@JsonIgnore:** exclude a field from serialization
- **@JsonFormat:** control date and time formatting
- **@JsonInclude(NON_NULL):** omit null fields from the response



REQUEST AND RESPONSE DTOs

- **CustomerRequest:** firstName, lastName, email -- with validation annotations
- **CustomerResponse:** customerId, firstName, lastName, email, status
- Sensitive or internal fields (internalRiskFlag, createdAt) are never exposed



WHY SEPARATE THE TWO MODELS

- The database schema can change without breaking the public API contract
- The API contract can add fields without touching persistence
- Internal-only fields (audit flags, soft-delete markers) never leak into a response

KEY TAKEAWAY Spring Boot wires Jackson in automatically with Spring Web -- annotations let developers control field names, dates, and null handling. • A CustomerRequest and CustomerResponse each expose only the fields a client needs -- never the full internal entity. • Keeping the JPA entity and the API DTO as two separate classes lets each evolve independently without breaking the other.

MAPPING BETWEEN ENTITIES AND DTOS · INPUT VALIDATION WITH JAKARTA VALIDATION

COMMON VALIDATION ANNOTATIONS

ANNOTATION	PURPOSE
@NotNull / @NotBlank	Value cannot be null / cannot be null or blank
@Size	String or collection size
@Email	Email format
@Min / @Max	Numeric range
@Pattern	Regular expression
@Past / @Future	Date must be in the past / future

MAPPING APPROACHES

- **Manual mapping:** a static from() factory method on the DTO
- **Mapping libraries:** MapStruct generates mapper implementations at compile time
- Whichever approach is chosen, apply it consistently across the service

KEY TAKEAWAY Manual factory methods or a mapping library like MapStruct both work -- the goal is one consistent, tested conversion path per direction. • @NotBlank, @Email, @Size, and @Valid on the controller method turn field-level rules into automatic 400 Bad Request responses.

CUSTOM VALIDATION RULES · CENTRALIZED EXCEPTION HANDLING



WHEN TO WRITE A CUSTOM VALIDATOR

- A rule spans multiple fields, e.g. start date must be before end date
- A rule requires a database lookup, e.g. checking whether an email already exists
- Database-backed checks belong in the service layer, not in a Bean Validation annotation



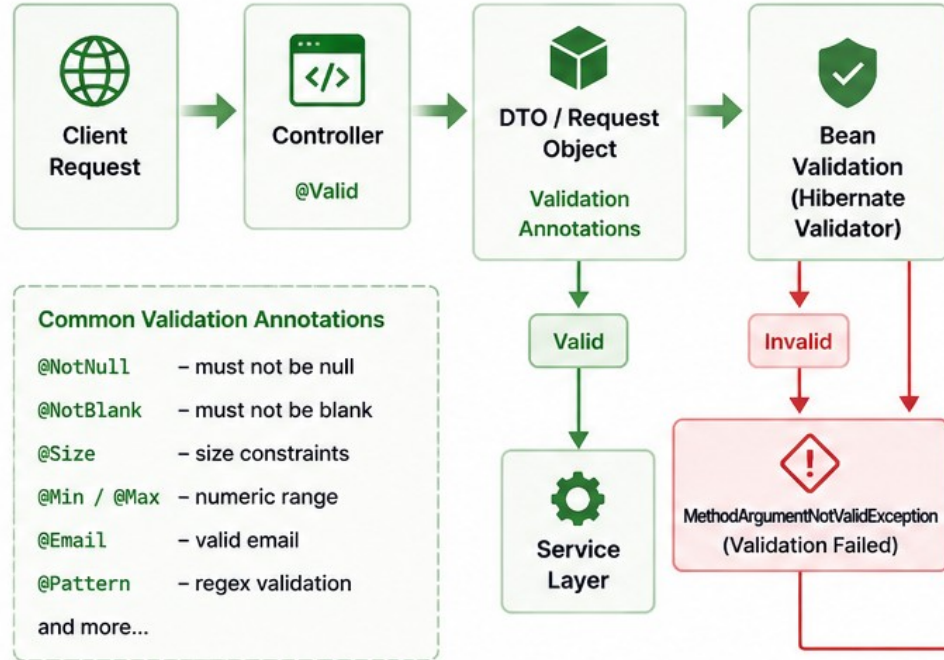
GLOBAL EXCEPTION HANDLER EXAMPLE

- @RestControllerAdvice class centralizes all @ExceptionHandler methods
- @ExceptionHandler(CustomerNotFoundException.class) maps to 404 Not Found
- Removes the need to duplicate try/catch logic in every controller

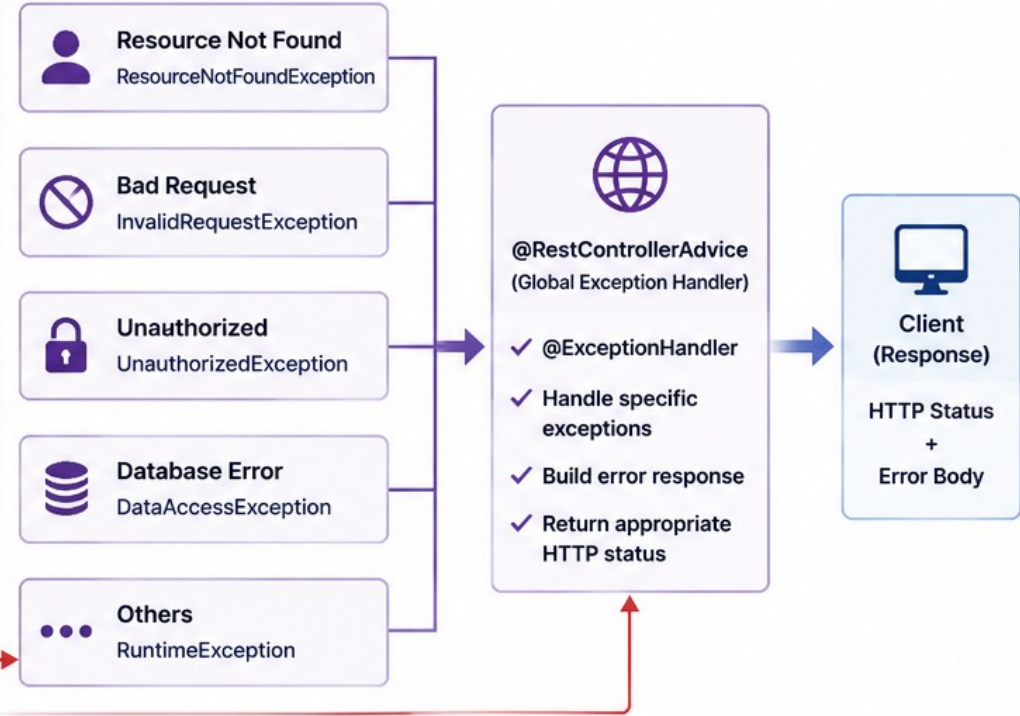
KEY TAKEAWAY A custom @Constraint annotation plus a ConstraintValidator implementation lets teams enforce domain-specific rules declaratively. • One GlobalExceptionHandler class maps every custom exception to a consistent status code and error body across the whole service.

Validation and Exception Handling

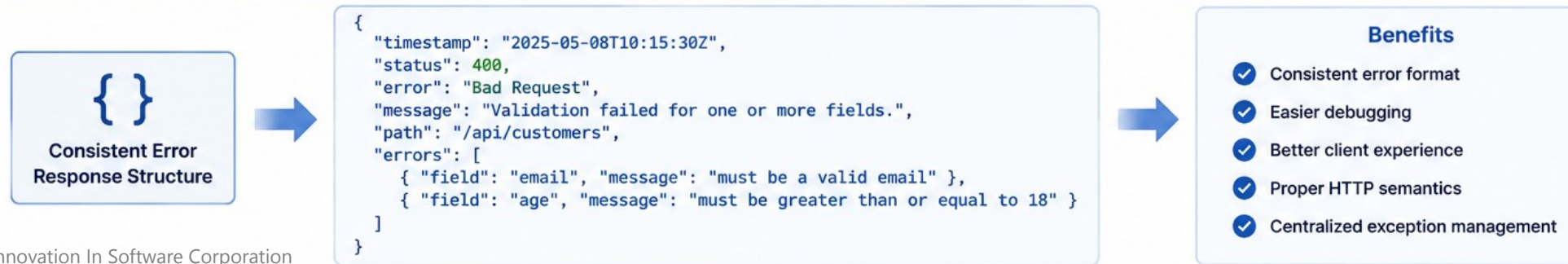
1. Validation Flow



2. Exception Handling Flow



3. Standard Error Response



DESIGNING CONSISTENT ERROR RESPONSES

code, message, timestamp, and path in every error response turn a scattered set of failures into one predictable contract.



STANDARD ERROR RESPONSE FIELDS

- **code:** CUSTOMER_NOT_FOUND -- a stable, application-specific identifier
- **message:** a clear, safe description of what went wrong
- **timestamp and path:** when the error happened and which endpoint triggered it

KEY TAKEAWAY code, message, timestamp, and path in every error response turn a scattered set of failures into one predictable contract.

Standardized API Error Responses

1. Purpose & Principles



Consistency

Same error format across all APIs and services



Clarity

Clear, human-readable messages



Debuggability

Easy to diagnose and resolve issues



Safety

Safe to expose; no sensitive information leakage

2. Standard Error Response Structure (JSON)

```
{  "timestamp": "2025-05-08T10:15:30Z",  "status": 400,  "error": "Bad Request",  "code": "VALIDATION_ERROR",  "message": "Validation failed for one or more fields.",  "path": "/api/customers",  "traceId": "4f8b7c2e9a3d4b11",  "errors": [    {      "field": "email",      "message": "must be a valid email"    },    {      "field": "age",      "message": "must be greater than or equal to 18"    }  ]}
```

Annotations for the JSON structure:

- timestamp**: When the error occurred
- status**: HTTP status code
- error**: Short description
- code**: Application-specific error code
- message**: Detailed message
- path**: Request path
- traceId**: Unique request/trace id
- errors**: Field-level validation errors

3. Common Status Code Mapping

Status Code	When to Use
400 Bad Request	Invalid request, validation failed
401 Unauthorized	Authentication required or failed
403 Forbidden	Authenticated but not authorized
404 Not Found	Requested resource not found
409 Conflict	Conflict with current state of resource
422 Unprocessable Entity	Semantically correct request with errors
500 Internal Server Error	Unexpected server error

4. Examples

400 Bad Request (Validation Error)

```
{  "timestamp": "2025-05-08T10:15:30Z",  "status": 400,  "error": "Bad Request",  "code": "VALIDATION_ERROR",  "message": "Validation failed for one or more fields.",  "path": "/api/customers",  "traceId": "4f8b7c2e9a3d4b11",  "errors": [    {      "field": "email",      "message": "must be a valid email"    }  ]}
```

401 Unauthorized

```
{  "timestamp": "2025-05-08T10:16:05Z",  "status": 401,  "error": "Unauthorized",  "code": "UNAUTHORIZED",  "message": "Authentication is required to access this resource.",  "path": "/api/orders",  "traceId": "7d9e2a7f6c1b4d23"}
```

404 Not Found

```
{  "timestamp": "2025-05-08T10:16:45Z",  "status": 404,  "error": "Not Found",  "code": "RESOURCE_NOT_FOUND",  "message": "Customer not found",  "path": "/api/customers/12345",  "traceId": "9a1c3d5e7b8f4a91"}
```

409 Conflict

```
{  "timestamp": "2025-05-08T10:17:20Z",  "status": 409,  "error": "Conflict",  "code": "EMAIL_ALREADY_EXISTS",  "message": "A customer with this email already exists.",  "path": "/api/customers",  "traceId": "2b6d4f8a9c0e7b33"}
```

500 Internal Server Error

```
{  "timestamp": "2025-05-08T10:18:10Z",  "status": 500,  "error": "Internal Server Error",  "code": "INTERNAL_SERVER_ERROR",  "message": "An unexpected error occurred. Please try again later.",  "path": "/api/payments",  "traceId": "f3e7b2c1d9a44f77"}
```



Best Practices



Use consistent field names



Provide meaningful error codes



Include traceId for correlation



Return only safe information



Log full details on server



Version your error schema if needed

USING RESPONSEENTITY EFFECTIVELY · IMPLEMENTING PAGINATION AND SORTING · API VERSIONING IN SPRING BOOT



WHY USE RESPONSEENTITY

- `ResponseEntity.status(HttpStatus.CREATED).body(response)` for POST
- `ResponseEntity.noContent().build()` for a successful DELETE
- Full control over status code, headers (e.g. Location), and body together



PAGINATION WITH SPRING DATA

- Controller accepts Pageable from query parameters (page, size, sort)
- **Repository method:** `Page<Customer> findAll(Pageable pageable)`
- Response includes content plus page, size, totalElements, and totalPages



IMPLEMENTING VERSIONING IN SPRING BOOT

- **URI:** `@RequestMapping("/v1/customers")` on a versioned controller
- **Header:** `@GetMapping(headers = "X-API-Version=2")`
- Keep the versioning strategy consistent across every microservice in the platform

KEY TAKEAWAY Returning a plain object hides HTTP intent; `ResponseEntity.status(HttpStatus.CREATED).body(response)` makes it explicit. • `Page<Customer> findAll(Pageable pageable)` returns page content plus metadata like `totalElements` and `totalPages`, ready for the client. • Whichever strategy a service chooses, applying it consistently across all Spring Boot microservices matters more than which one is picked.

OpenAPI and Swagger Integration

Automatically document and visualize your Spring Boot REST APIs.



Goal: Provide interactive API documentation for developers and consumers using the OpenAPI specification and Swagger UI.



Standardized
API Contract



Auto-Generated
Documentation



Interactive
Explorer



Up-to-Date
and Consistent



Better Developer
Experience

1. Add Dependencies



```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.3.0</version>
</dependency>
```



```
implementation 'org.springdoc:springdoc-openapi-starter-webmvc-ui:2.3.0'
```



springdoc-openapi-starter-webmvc-ui provides both OpenAPI generation and Swagger UI.

2. Basic Configuration

```
application.yml
springdoc:
  api-docs:
    path: /v3/api-docs
  swagger-ui:
    path: /swagger-ui.html
    operationsSorter: method
    tagsSorter: alpha
  info:
    title: User Service API
    description: API for managing users
    version: 1.0.0
    contact:
      name: API Support
      email: support@myapp.com
    license:
      name: Apache 2.0
      url: https://www.apache.org/licenses/LICENSE-2.0
```

3. Example API with Annotations

```
@RestController
@RequestMapping("/api/users")
@Tag(name = "Users", description = "User management APIs")
public class UserController {

    @Operation(summary = "Get user by ID", description = "Returns a user by ID")
    @ApiResponse(responseCode = "200", description = "User found"),
    @ApiResponse(responseCode = "404", description = "User not found")
    @GetMapping("/{id}")
    public ResponseEntity<UserDTO> getUser(@PathVariable Long id) {
        UserDTO user = userService.getUserById(id);
        return ResponseEntity.ok(user);
    }
}
```

4. Access Points



OpenAPI JSON

<http://localhost:8080/v3/api-docs>



OpenAPI YAML

<http://localhost:8080/v3/api-docs.yaml>



Swagger UI

<http://localhost:8080/swagger-ui.html>



These endpoints are auto-configured by **springdoc-openapi**.

5. Swagger UI (Interactive Documentation)

6. Best Practices

- ✓ Keep API documentation consistent with implementation.
- ✓ Use meaningful summaries and descriptions.
- ✓ Group APIs using `@Tag` for better organization.
- ✓ Document request/response models clearly.
- ✓ Include examples for complex payloads.
- ✓ Secure docs in production if necessary.
- ✓ Version your API and document each version.



Tip: Use profiles to disable Swagger UI in production:
`springdoc.swagger-ui.enabled=false`



Key Takeaway

OpenAPI + Swagger provide a single source of truth for your API contract, improving collaboration, testing, and adoption.



Remember: Great APIs are self-explanatory. Good documentation makes them easy to use.



GENERATING INTERACTIVE API DOCUMENTATION · API TESTING WITH POSTMAN AND CURL · SPRING BOOT ACTUATOR INTRODUCTION



WHAT SWAGGER UI PROVIDES

- A live, interactive page listing every endpoint, grouped by controller
- Try-it-out execution of real requests directly from the browser
- Request/response schemas, example payloads, and status codes on one page



TESTING WITH CURL

- `curl -X GET "http://localhost:8080/api/v1/users/1" -H "Accept: application/json"`
- `curl -X POST ... -d '{"name":"Charlie","email":"charlie@example.com"}'`



TESTING WITH POSTMAN

- Organize requests into Collections; use Environments for base URLs and variables
- Add Tests (JS scripts) to validate status codes and response bodies automatically



WHAT ACTUATOR PROVIDES

- Built-in endpoints exposing operational information about the application
- Health monitoring, metrics collection, auditing, and runtime management
- Designed for production environments with security and minimal performance impact

KEY TAKEAWAY Developers and testers can call endpoints, inspect schemas, and see live example responses without leaving the browser. • Fast, easy request sending, response validation, and automated test scripts make Postman and cURL essential day-to-day dev tools. • Adding spring-boot-starter-actuator gives every service health, metrics, info, and management endpoints out of the box.

Actuator Introduction

1. What is Actuator?

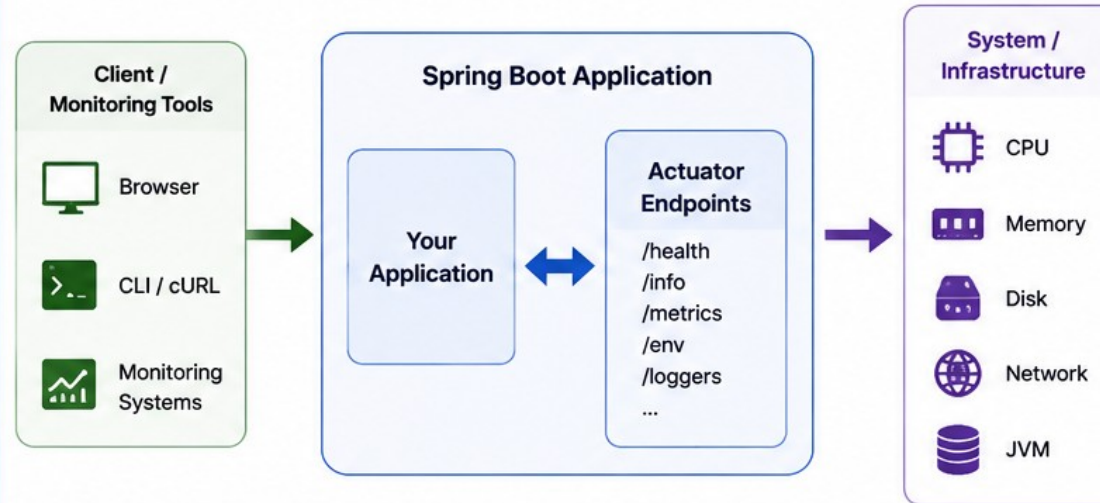


Spring Boot Actuator

provides production-ready features to help you monitor and manage your application.

- ✓ Built-in Endpoints
- ✓ Operational Insights
- ✓ Health Monitoring
- ✓ Metrics & Monitoring
- ✓ Easy Integration

2. How Actuator Works



3. Key Benefits

- ✓ **Production Ready**
Monitor and manage in production environments.
- ✓ **Observability**
Gain insights into application health and performance.
- ✓ **Troubleshooting**
Quickly identify and resolve issues.
- ✓ **Integration**
Easily integrates with monitoring tools (Prometheus, Grafana, ELK, etc.).
- ✓ **Customizable**
Enable/disable endpoints as per requirements.

4. Common Endpoints

Endpoint	Description
/health	Application health information
/info	Application information
/metrics	Application metrics
/env	Environment properties
/loggers	View and modify log levels
/beans	Spring beans information
/threaddump	Thread dump
/heapdump	Heap dump
/prometheus	Metrics for Prometheus

5. Enabling Actuator

Add Dependency

Maven

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Gradle

```
implementation 'org.springframework.boot:' +
'spring-boot-starter-actuator'
```

Enable Endpoints

application.properties / application.yml

```
management.endpoints.web.exposure.include=*
# or specific endpoints
# management.endpoints.web.exposure.include=
health,info,metrics
```

```
management.endpoint.health.show-details=always
management.endpoints.web.base-path=/actuator
```

6. Access Actuator Endpoints

Default Access

 <http://localhost:8080/actuator/health>

Custom Base Path (optional)

```
management.endpoints.web.base-path=/manage
Access: http://localhost:8080/manage/health
```

Security (recommended)



Secure Actuator endpoints in production using Spring Security.

HEALTH, INFO, AND METRICS ENDPOINTS · SECURING MANAGEMENT ENDPOINTS



THREE CORE ENDPOINTS

- **/actuator/health:** aggregated UP/DOWN status, used by Kubernetes liveness and readiness probes
- **/actuator/info:** application metadata, build information, and environment details
- **/actuator/metrics:** JVM, HTTP, and custom business metrics, often in Prometheus format



SECURING ACTUATOR IN PRODUCTION

- Expose only the endpoints you need via `management.endpoints.web.exposure.include`
- Require authentication with Spring Security, role-based access, or mutual TLS
- Consider isolating management traffic on a separate port (`management.server.port`)

KEY TAKEAWAY /actuator/health feeds Kubernetes probes, /actuator/info carries build metadata, and /actuator/metrics powers dashboards and alerting. • Expose only what's needed, require authentication, and consider a separate management port to isolate operational traffic.

PRODUCTION-READY REST API CHECKLIST · ENVIRONMENT-SPECIFIC SERVICE CONFIGURATION



CHECKLIST -- SECTIONS 1 TO 4

- 1. Design & standards:** REST principles, consistent naming, HTTPS everywhere
- 2. Security:** authentication, authorization, input validation, rate limiting
- 3. Contracts & documentation:** OpenAPI spec accurate and current
- 4. Implementation:** consistent errors, centralized exception handling, DTOs not entities



ACTIVATING PROFILES AT RUNTIME

- **Via application.properties:** `spring.profiles.active=dev`
- **Via command line:** `java -jar orders-service.jar --spring.profiles.active=prod`
- **Via environment variable:** `SPRING_PROFILES_ACTIVE=prod`



CHECKLIST -- SECTIONS 5 TO 8

- 5. Performance & scalability:** pagination, caching, stateless design
- 6. Observability:** health, metrics, correlation IDs, distributed tracing
- 7. Reliability & resilience:** timeouts, retries, circuit breakers, idempotency
- 8. Operations & maintenance:** CI/CD, contract tests, secrets managed securely



BEST PRACTICES

- Keep environment-specific files under version control (without real secrets)
- Never hardcode values in code; validate configuration at startup (fail fast)

KEY TAKEAWAY Design, security, contracts, implementation, performance, observability, reliability, and operations -- eight lenses, one review gate. • Build once, configure per environment: the goal environment-specific configuration exists to achieve, without ever touching source code.

TYPE-SAFE CONFIGURATION PROPERTIES · CONFIGURATION VALIDATION · MANAGING SENSITIVE CONFIGURATION



HOW IT WORKS

- Define an `@Validated @ConfigurationProperties(prefix = "app.orders")` class with Jakarta constraints
- Enable it with `@EnableConfigurationProperties` on the application class
- Inject and use it as a plain, strongly typed Java object anywhere it's needed



WHAT NOT TO DO

- Hardcode secrets in application code or commit them to `application.yml`
- Log or expose secret values in error messages



FAIL FAST ON BAD CONFIGURATION

- `@NotBlank`, `@Min`, `@Max`, `@DurationMin` constraints on configuration properties
- Invalid values produce a clear "APPLICATION FAILED TO START" error naming the exact field
- Catches misconfiguration in minutes during deployment, not hours later in production



HOW TO PROVIDE SECRETS SECURELY

- Environment variables, Spring Cloud Config, HashiCorp Vault, or OpenShift Secrets
- Always scope access to the minimum required (least privilege)

KEY TAKEAWAY `@Validated @ConfigurationProperties` classes bring compile-time safety, IDE auto-completion, and constraint validation to configuration. • A missing or invalid property should stop the application from starting with a clear message, not surface as a mysterious runtime bug. • Passwords, API keys, tokens, and certificates should be externalized as placeholders and injected from a vault or platform secret store.

Logging Basics and Sensitive-Data Caution

1. Why Logging?



Observability

Understand application behavior and flow.



Troubleshooting

Diagnose issues and root causes.



Audit & Compliance

Track important events and user actions.



Security Monitoring

Detect suspicious and unauthorized activities.



Performance Insight

Monitor performance and bottlenecks.

2. Log Levels

Level	Use When	Example
TRACE	Very detailed information for debugging.	Method entry/exit, variable values
DEBUG	Detailed debugging information.	SQL queries, API requests
INFO	General operational information.	Application started, User logged in
WARN	Potential issues that don't prevent execution.	Retry attempts, Deprecated usage
ERROR	Errors that prevent execution.	Unhandled exception, DB connection failed
FATAL	Critical errors causing application failure.	OutOfMemoryError, System crash

Less Detailed

More Severe

3. Logging Best Practices

- ✓ **Log at the right level.**
Avoid using ERROR for normal flow.
- ✓ **Be clear and consistent.**
Use meaningful messages and formats.
- ✓ **Include context.**
Add IDs, usernames, request IDs, etc.
- ✓ **Avoid log noise.**
Don't log too much or too often.
- ✓ **Use structured logging.**
Prefer JSON logs for better analysis.
- ✓ **Monitor logs.**
Centralize and alert on important logs.

4. What to Log vs What NOT to Log



Log This

- ✓ Request ID, Timestamp
- ✓ User ID / Account ID
- ✓ API endpoint, HTTP method
- ✓ Business events (e.g., order created)
- ✓ Validation failures (without sensitive data)
- ✓ Error messages and stack traces (without sensitive data)
- ✓ Performance metrics



Do NOT Log This

- ✗ Passwords / Plaintext credentials
- ✗ Tokens (JWT, OAuth, API keys)
- ✗ Credit card numbers / CVV
- ✗ Personal Identifiable Information (PII) (SSN, PAN, Aadhaar, etc.)
- ✗ Bank account numbers / IFSC
- ✗ Secrets, private keys, certificates
- ✗ Sensitive business data

5. Sensitive-Data Caution



Sensitive data in logs can lead to security breaches, compliance violations, and loss of trust.

Example: What NOT to Log

```
{ "event": "login", "user": "john.doe",  
  "password": "P@ssw0rd123",  
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "cardNumber": "4111-1111-1111-1111",  
  "cvv": "123" }
```



Example: Safe Logging

```
{ "event": "login", "userId": "u12345", "status": "success",  
  "ip": "192.168.1.10", "requestId": "req-8f3a2c" }
```



How to Protect



Mask sensitive data (e.g., ****1234)



Use filters / appenders to exclude data



Encrypt sensitive data if necessary



Follow compliance policies (GDPR, PCI-DSS, etc.)

LOGGING WITH SLF4J AND LOGBACK

Code logs against the SLF4J facade; Logback handles console, rolling file, and other appenders underneath without code changes.

LOG LEVELS

- **TRACE:** most detailed diagnostic information
- **DEBUG:** developer troubleshooting **INFO:** normal application events
- **WARN:** unexpected but recoverable **ERROR:** failures that require attention

BEST PRACTICES

- **Use parameterized logging:** `log.info("Creating order with id={}", id)`
- Never log passwords, tokens, or PII

KEY TAKEAWAY Code logs against the SLF4J facade; Logback handles console, rolling file, and other appenders underneath without code changes.

STRUCTURED LOGGING · CORRELATION IDS ACROSS SERVICES



UNSTRUCTURED VS. STRUCTURED

- **Unstructured:** free-text log lines that are hard to parse or query
- **Structured:** {"timestamp":..., "level":"ERROR", "service":"order-service", "orderId":"12345", ...}



COMMON STRUCTURED FIELDS

- timestamp, level, service, event, userId, orderId/resourceId, correlationId/traceId, message



HOW TO IMPLEMENT IT IN SPRING BOOT

1. Generate or extract the correlation ID at the edge (gateway or controller filter)
2. Propagate it to MDC, outgoing requests, messages, and tracing context
3. Log with MDC so every log pattern includes the correlation ID
4. Always clear MDC after the request completes to avoid leaking context

KEY TAKEAWAY A structured log line with service, event, orderId, and correlationId fields can be filtered, alerted on, and correlated at scale. • Client -> API Gateway -> Order Service -> Payment Service -> Inventory Service, all logging the same ID, all traceable as one request.

EFFECTIVE LOG-LEVEL MANAGEMENT · DESIGNING SERVICE AND REPOSITORY LAYERS

📌 RESPONSIBILITIES AT A GLANCE

LAYER	RESPONSIBILITY
Controller	Map HTTP requests, validate/transform input, return responses
Service	Implement use cases, manage transactions, coordinate repositories
Domain	Encapsulate business rules and invariants
Repository	Abstract data access, provide CRUD and query operations

📌 ENVIRONMENT-BASED STRATEGY

- **Local dev:** DEBUG/TRACE Test/QA: INFO/DEBUG
- **Staging:** INFO/WARN Production: INFO/WARN/ERROR

✅ DYNAMIC LOG-LEVEL CONTROL

- **POST /actuator/loggers/{name} {"configuredLevel": "DEBUG"}**
- Great for on-demand troubleshooting without a redeploy -- revert once resolved

KEY TAKEAWAY More verbosity in lower environments, less in production -- with dynamic Actuator-based overrides for on-demand troubleshooting.

- The dependency rule -- higher layers depend on lower layers, never the reverse -- is what keeps a layered microservice loosely coupled.

FEATURE FLAGS AND RUNTIME BEHAVIOR

Dark launches, gradual rollouts, and instant kill switches all become possible once behavior is controlled by a flag instead of a deployment.



WHY USE FEATURE FLAGS

- **Safer releases:** deploy code disabled, enable only when ready
- **Gradual rollouts:** 5% -> 25% -> 50% -> 100% of users, monitoring metrics at each step
- **Quick rollback:** disable a feature instantly without a code change or redeploy

KEY TAKEAWAY Dark launches, gradual rollouts, and instant kill switches all become possible once behavior is controlled by a flag instead of a deployment.

Running, Packaging, and Unit-Test Templates

1. Running the Application

A Using IDE



- Open the project in IntelliJ IDEA
- Run the main class (@SpringBootApplication)
- Click ▶ Run

B Using Maven



```
./mvnw spring-boot:run  
# or  
mvn spring-boot:run
```

C Using Gradle



```
./gradlew bootRun  
# or  
gradle bootRun
```

D Directly (from JAR)



```
java -jar build/libs/  
your-app-0.0.1-SNAPSHOT.jar
```

Default Paccess



<http://localhost:8080>

2. Packaging the Application

A Maven



Maven

```
./mvnw clean package  
# or  
mvn clean package
```



target/your-app-0.0.1-SNAPSHOT.jar

B Gradle



Gradle

```
./gradlew clean build  
# or  
gradle clean build
```



Output
build/libs/your-app-0.0.1-SNAPSHOT.jar



The generated JAR is executable and contains an embedded server.

3. Unit-Test Templates

A Service Layer Test



Service

```
@ExtendWith(MockitoExtension.class)  
class UserServiceTest {  
    @Mock  
    private UserRepository repo;  
  
    @InjectMocks  
    private UserService service;  
  
    @Test  
    void shouldReturnUserById() {  
        when(repo.findById(1L)).thenReturn(  
            Optional.of(new User(1L, "John"))  
        );  
  
        User user = service.getId(1L);  
  
        assertEquals("John", user.getName());  
        verify(repo).findById(1L);  
    }  
}
```

B Controller Layer Test (WebMvcTest)



Controller

```
@WebMvcTest(UserController.class)  
class UserControllerTest {  
    @Autowired  
    private MockMvc mockMvc;  
  
    @MockBean  
    private UserService service;  
  
    @Test  
    void shouldReturnUser() throws Exception {  
        when(service.getId(1L)).thenReturn(  
            new User(1L, "John")  
        );  
  
        mockMvc.perform(get("/api/users/1"))  
            .andExpect(status().isOk())  
            .andExpect(jsonPath("$.name").value("John"));  
    }  
}
```

C Repository Layer Test (DataJPA)



Repository

```
@DataJpaTest  
class UserRepositoryTest {  
    @Autowired  
    private UserRepository repo;  
  
    @Test  
    void shouldFindByEmail() {  
        User u = new User(null, "test@example.com", "John");  
        repo.save(u);  
  
        Optional<User> found = repo.findByEmail("test@example.com");  
        assertTrue(found.isPresent());  
    }  
}
```

Best Practices



Use `spring-boot:run` for quick development.



Use the packaged JAR for staging/production.



Always run tests before packaging.



Keep tests fast, isolated and reliable.

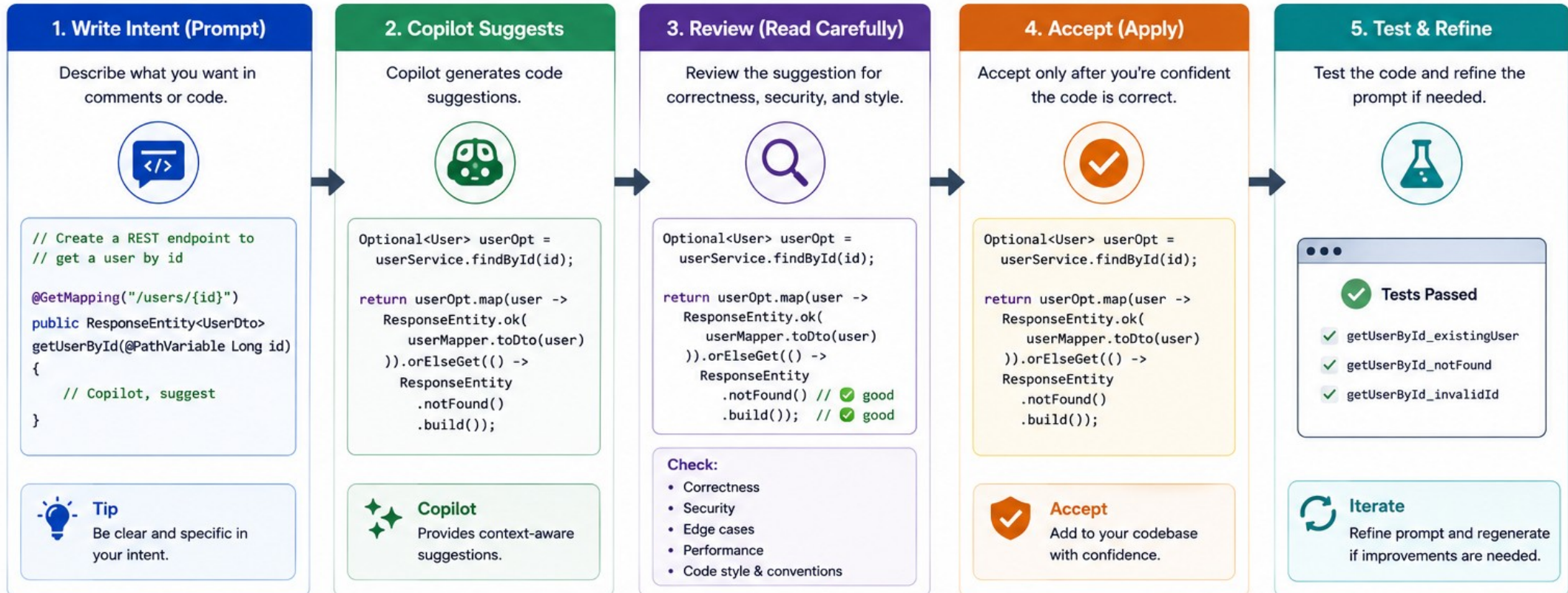


Aim for good test coverage.



Use `@TestConfiguration` for test-specific beans.

GitHub Copilot Demonstration (review-before-accept)



Why Use GitHub Copilot?

 Boost Productivity Write code faster and focus on important logic.	 Context-Aware Understands your code, comments, and project.	 Reduce Boilerplate Generates repetitive code and common patterns.	 Learn as You Code Discover new APIs, best practices, and patterns.	 Maintain Control You review and decide what goes into your code.
--	---	---	--	--

Review-Before-Accept Best Practices

- ✓ **Never blindly accept.**
Always read and understand the suggested code.
- ✓ **Validate logic & edge cases.**
Ensure it handles errors, nulls, and boundary conditions.
- ✓ **Check security implications.**
Avoid vulnerabilities like injection, insecure data handling, etc.
- ✓ **Follow project standards.**
Ensure naming, structure, and style match your codebase.
- ✓ **Test the changes.**
Run unit/integration tests before moving on.

Golden Rule

Copilot is your pair programmer, you are the code owner.

SEPARATION OF CONCERNS · TRANSACTION BOUNDARIES IN SERVICE CODE



WITHOUT SEPARATION OF CONCERNS

- Hard to understand, difficult to test, changes break many parts



WITH SEPARATION OF CONCERNS

- Clear boundaries and responsibilities, easier unit testing
- Changes are isolated, the system is maintainable and scalable



KEY PRINCIPLES

- Place transaction boundaries at the service layer, not controllers or repositories
- One business operation, one transaction -- align it to a single use case
- Keep transactions short; avoid remote calls and long computations inside them
- Do not span multiple services -- use messaging, the outbox pattern, or sagas instead

KEY TAKEAWAY Without separation of concerns, every change risks breaking something unrelated; with it, changes stay isolated and testable. • @Transactional belongs on the service layer, scoped to one business operation, and should stay short and free of remote calls.



Reusable Components Without Tight Coupling

Build Once, Use Many — Keep Services Independent



What We Want

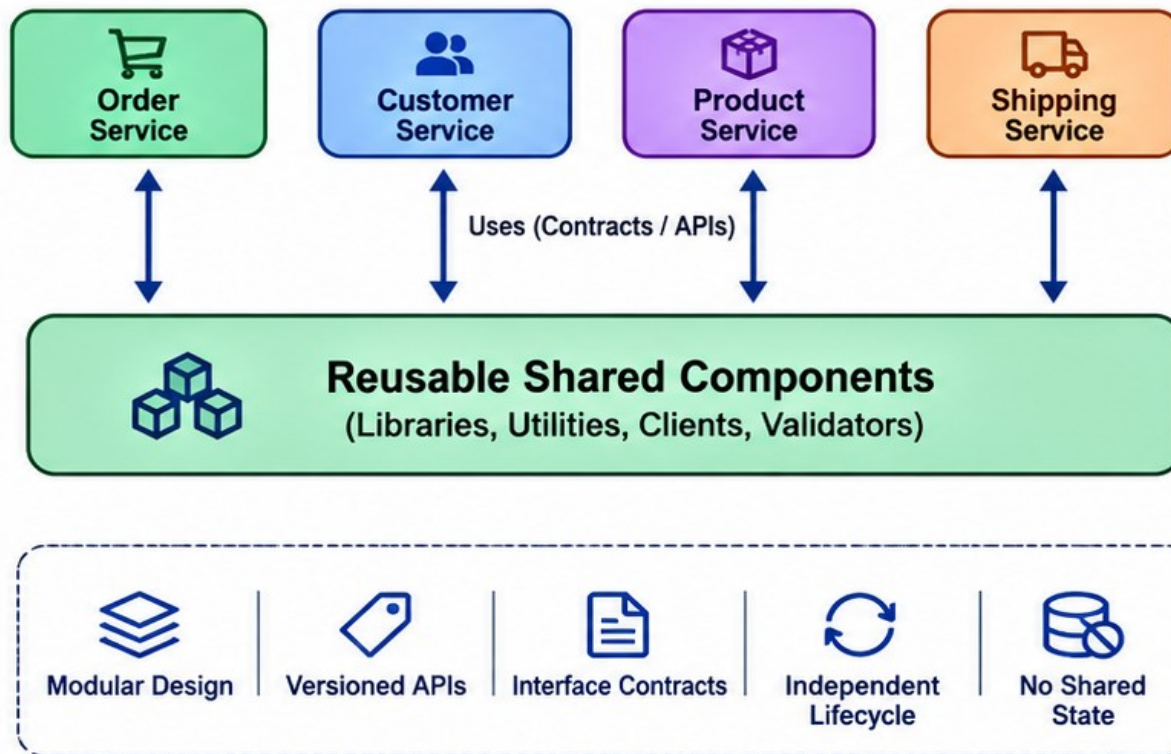
- ✓ Reusable components with clear contracts
- ✓ Loose coupling between components
- ✓ Independent evolution and deployment
- ✓ High cohesion within each component
- ✓ Minimal ripple effects from change



Tight Coupling Anti-Patterns

- ✗ Direct component dependencies
- ✗ Shared mutable state
- ✗ Calling internal classes directly
- ✗ Implicit contracts
- ✗ Changes requiring multiple updates

Loose Coupling, High Reusability



Best Practices

- ✓ Define clear, stable interfaces
- ✓ Use dependency injection
- ✓ Publish reusable libraries or APIs
- ✓ Version components explicitly
- ✓ Avoid shared mutable state
- ✓ Use messaging for loose integration
- ✓ Keep components independent



Results

- ✓ Greater reusability
- ✓ Easier testing
- ✓ Independent deployment
- ✓ Higher maintainability
- ✓ Faster innovation



Key Takeaway: Reusable components deliver maximum value when they are designed with clear contracts and loose coupling, enabling independent evolution and dependable microservices.

MANAGING THIRD-PARTY DEPENDENCIES

Evaluating, pinning, scanning, and regularly updating dependencies is how a team owns the risk that comes with using external code.



EIGHT BEST PRACTICES

1. **Evaluate before adding:** check licence, maintenance activity, and whether the standard library already covers it
2. **Use official sources:** pull from Maven Central or an approved internal mirror, never an untrusted copy
3. **Pin versions:** declare exact versions so every build and environment resolves the same artifacts
4. **Manage transitive dependencies:** use dependencyManagement or a BOM so indirect versions cannot drift
5. **Scan continuously:** run vulnerability scanning in the pipeline, not as an occasional manual review
6. **Update regularly:** small frequent upgrades are far cheaper than a forced jump several major versions later
7. **Review licenses:** confirm each licence is compatible with how the bank distributes and runs the software
8. **Remove unused dependencies:** every unused library is attack surface and build time you gain nothing from

KEY TAKEAWAY Evaluating, pinning, scanning, and regularly updating dependencies is how a team owns the risk that comes with using external code.

SPRING BOOT CODING STANDARDS · MICROSERVICE PACKAGE ORGANIZATION



CORE CODING GUIDELINES

- Use constructor injection; keep controllers thin, business logic in services
- Validate input with @Valid; handle exceptions with @ControllerAdvice
- Use meaningful logging and records for immutable DTOs where appropriate



GOOD: FEATURE-ORIENTED

- orders/, customers/, payments/ -- each with its own controller, service, domain, repository, dto



AVOID: LAYERED (TECHNICAL)

- One controller/ package, one service/ package, one repository/ package for the whole app
- Structure should reflect the domain, not the technology

KEY TAKEAWAY Consistency, clarity, simplicity, cohesion, and testability are the five principles behind every Spring Boot coding guideline. • A feature-oriented package (orders/controller, orders/service, orders/repository) scales far better than one giant controller package.

CLEAN ARCHITECTURE IN SPRING BOOT · HEXAGONAL ARCHITECTURE AND PORTS AND ADAPTERS



THE CLEAN ARCHITECTURE LAYERS

- **Entities:** core domain objects and business rules, no dependencies
- **Use cases:** application-specific business rules that orchestrate entities
- **Interface adapters:** controllers, gateways, presenters that convert data formats
- **Frameworks & drivers:** web, database, messaging, and external APIs



THE HEXAGON

- **Driving adapters (inbound):** REST API, messaging, CLI / batch jobs
- **Application core:** entities, value objects, domain services, business rules
- **Driven adapters (outbound):** database, external API, email service



THE DEPENDENCY RULE

- Source code dependencies can only point inward, toward the domain
- The core can be tested completely without any infrastructure



WHY IT MATTERS

- The core defines ports (interfaces); adapters implement those interfaces
- Swap a database or messaging technology without touching business logic
- The core can be tested in isolation with mocked ports and adapters

KEY TAKEAWAY Entities at the center, use cases around them, and interface adapters and frameworks on the outside -- dependencies only ever point inward. • REST, messaging, and CLI driving adapters, and database, external-API, and email driven adapters, all surround one protected application core.

EXERCISE 1.3 — SCAFFOLD ACCOUNT ENDPOINTS AND VALIDATION

Module 3 checkpoint C — Controllers, DTOs, validation rules.



OBJECTIVE

- Controllers, DTOs, validation rules
- 15 min · code warmup checkpoint
- **Worksheet:**
curriculum/day-01/exercises/exercise-1.3-scaffold-account.md



DO THIS

1. Open labs/day-01/lab1/starter and find controller, service, DTO, and validation TODOs
2. **Name the layers:** Controller → Service → Repository → PostgreSQL (Flyway, not Hibernate DDL)
3. **List validation rules you will enforce (customerId CUST- plus four digits, nickname length, legal status change). accountId is generated**
4. Confirm logs will use accountId/status only — no request bodies or PII

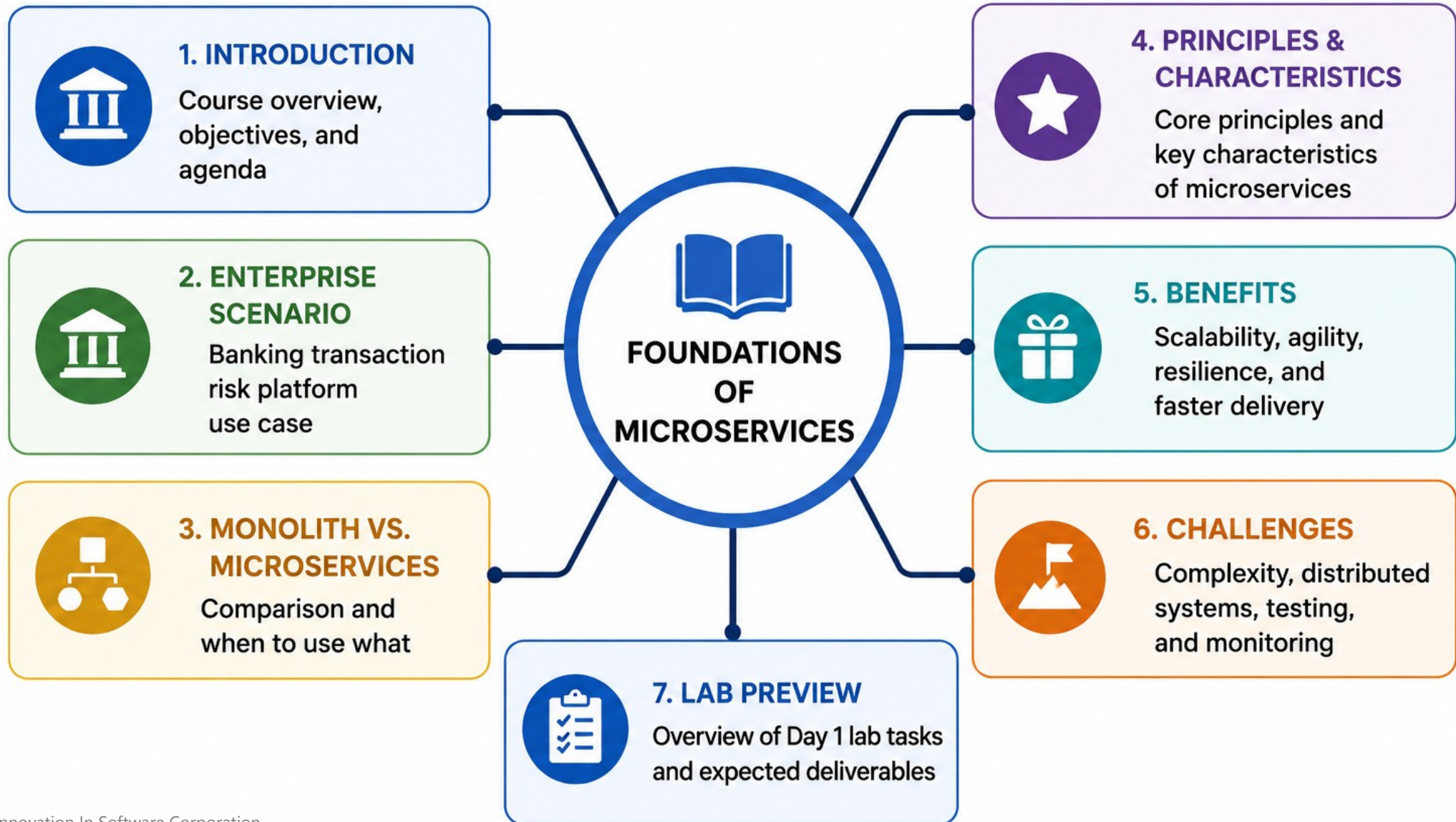


EXPECTED RESULT

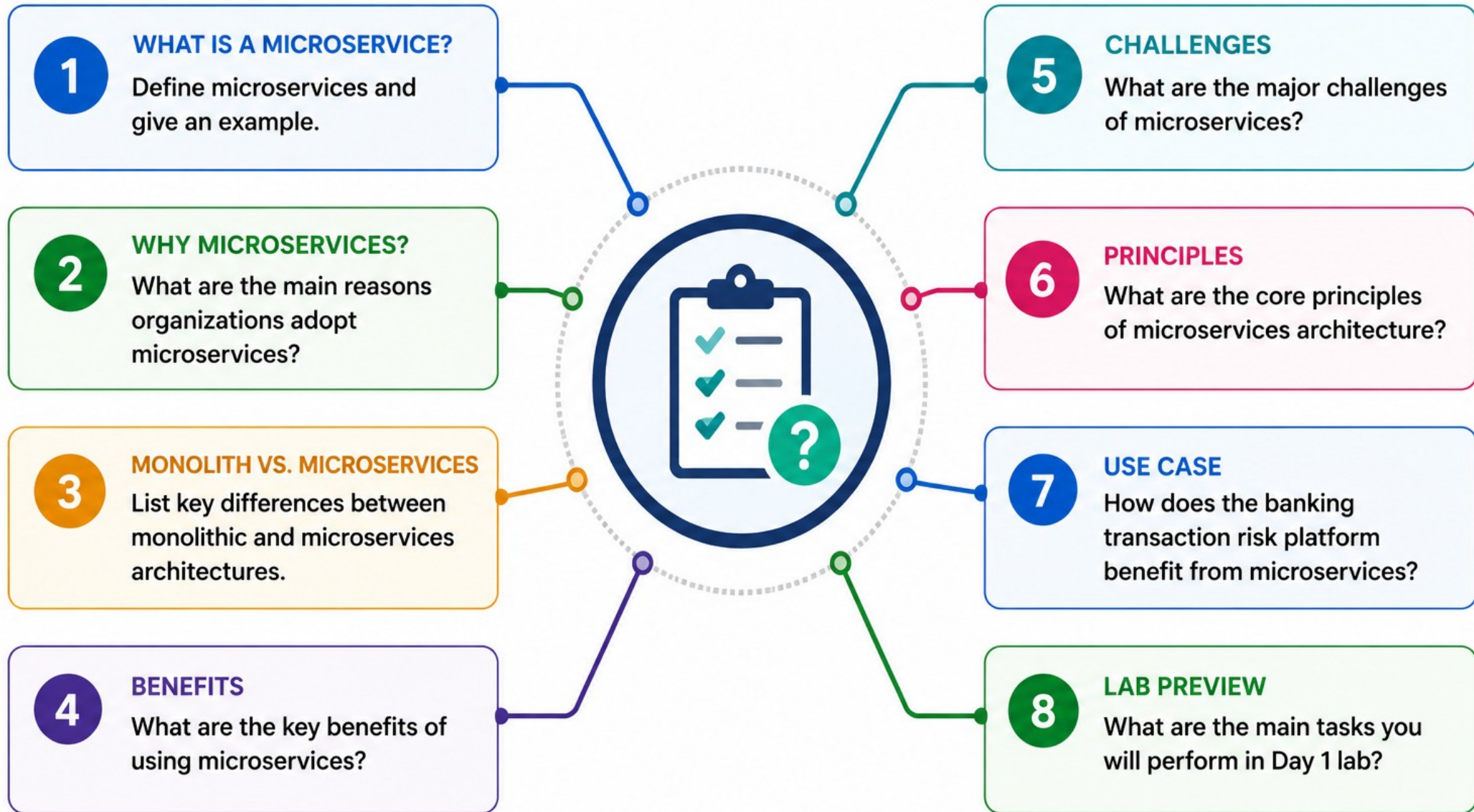
- You can point at the starter files Lab 1 will complete. Do not skip ahead into Lab 2.
- Finish the implementation in Lab 1 (starter on port 8081).

KEY TAKEAWAY You can point at the starter files Lab 1 will complete. Do not skip ahead into Lab 2.

DAY 1 SUMMARY



KNOWLEDGE CHECK



LAB 1

Lab 1 — Account Service

A functional, documented Spring Boot Account Service — capstone service 1 of 3.



Lab 1 Overview - Account Service

1. Purpose

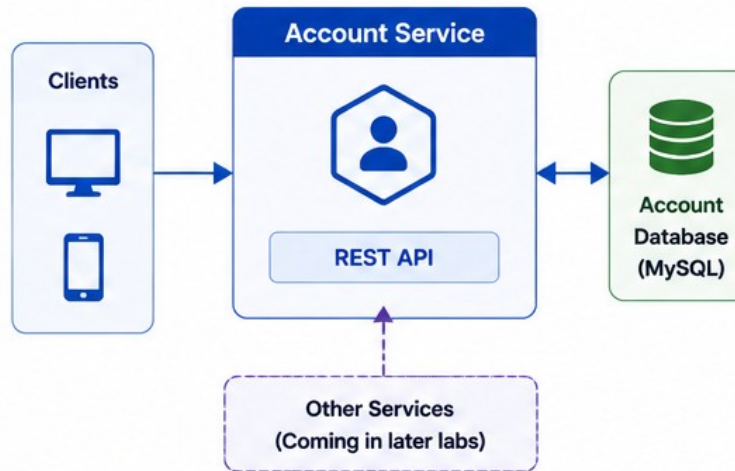


Build the Account Service, a foundational microservice that manages customer accounts.

You will learn:

- ✓ Create a Spring Boot REST API
- ✓ Design and implement JPA entities
- ✓ Expose CRUD endpoints
- ✓ Validate requests and handle errors
- ✓ Write unit tests
- ✓ Run and package the service

2. Account Service in the System



3. Key Features

- Manage Accounts**
Create, view, update, and delete accounts.
- Account Details**
Store account holder info and balance.
- Validation**
Ensure data integrity with request validation.
- Error Handling**
Standardized error responses.
- Testing**
Unit tests for service and controller layers.
- Production Ready**
Logging, configuration, and Actuator endpoints.

4. Tech Stack



Spring Boot 3.x
Web, Data JPA, Validation, Actuator



MySQL
Primary Database



Lombok
Reduce boilerplate code



JUnit 5 & Mockito
Unit testing



Maven
Build and dependency management

5. Account Entity

Field	Type	Description
id	Long	Primary Key
accountNumber	String	Unique account number
accountHolderName	String	Name of the account holder
email	String	Email of the account holder
accountType	String	e.g., SAVINGS, CHECKING
balance	BigDecimal	Account balance
createdAt	LocalDateTime	Record created timestamp
updatedAt	LocalDateTime	Record last updated timestamp

6. REST Endpoints

Method	Endpoint	Description
POST	/api/v1/accounts	Create a new account
GET	/api/v1/accounts	Get all accounts
GET	/api/v1/accounts/{id}	Get account by id
PUT	/api/v1/accounts/{id}	Update account by id
DELETE	/api/v1/accounts/{id}	Delete account by id

7. What You Will Deliver



REST API
with CRUD operations



JPA Entity &
Repository



Validation &
Exception Handling



Global Error
Responses

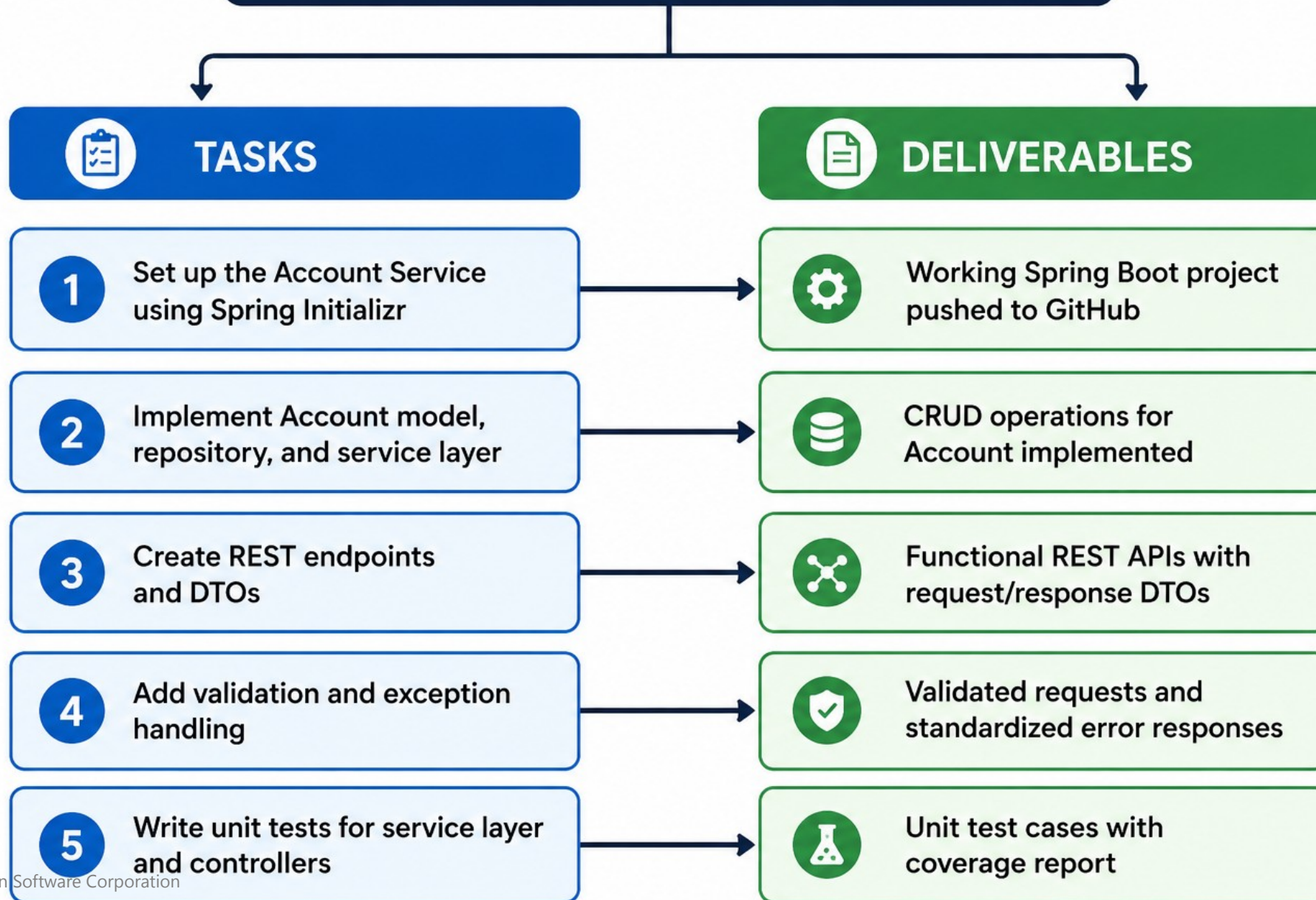


Unit Tests
(Service & Controller)



Executable JAR
(Maven Package)

LAB TASKS AND DELIVERABLES



LAB 1 — ACCOUNT SERVICE — OVERVIEW

Day 1 lab: what you build and how it grows the capstone.



WHAT YOU WILL FINISH WITH

- A Spring Boot 3 service running on port 8081
- A Flyway-managed accounts table in PostgreSQL
- Create, retrieve, and update operations
- **Approved status changes:** activate, freeze, close (no HTTP DELETE)
- Request validation and consistent error JSON
- Actuator health and Swagger UI
- Passing unit tests
- Logs that use synthetic identifiers only

KEY TAKEAWAY Lab 1 builds Account Service -- the working code behind today's concepts, and the next layer of the banking capstone.

LAB 1 — ACCOUNT SERVICE — BEFORE YOU START

Check these before the lab clock starts.



KNOWLEDGE YOU NEED (FROM DAY 1)

- Bounded context — this service owns accounts only, not transactions or risk
- Database per service — only Account Service talks to account_db
- REST resource — URLs name the account, not an action like /doActivate
- HTTP semantics — POST create, GET, PATCH nickname, POST .../activate
- Idempotency — activate already-ACTIVE or close already-CLOSED returns 200
- Twelve-Factor config — JDBC URL, user, and password in application-local.yml
- Layered Spring Boot — Controller → Service → Repository → PostgreSQL
- Flyway — SQL files version the schema; Hibernate does not create tables
- Validation + errors — bad input 400; missing account 404; illegal status 409
- Actuator + OpenAPI — health for operators; Swagger UI for consumers
- Log hygiene — log accountId and status; never bodies, emails, or PANs
- Synthetic IDs — customerId is CUST-0001; accountId is generated (ACC- plus hex)



ENVIRONMENT BASICS

- A clone of the public course repo (labs live under labs\)
- Java 21 (java -version should show 21)
- Maven 3.9+
- Docker Desktop started (the engine must be running, not only installed)
- Working directory: account-service (the folder with pom.xml)
- HTTP calls: curl.exe (not curl — PowerShell aliases curl)

KEY TAKEAWAY Settle prerequisites and environment first -- most lab time is lost to setup, not to code.

LAB 1 — ACCOUNT SERVICE — LAB STEPS

9 steps, worked in order.



LAB STEPS

- Step 1 — Run and validate the starter
- Step 2 — Apply the Flyway migration
- Step 3 — Confirm the entity and repository
- Step 4 — Complete service-layer state transitions
- Step 5 — Exercise the REST endpoints



LAB STEPS (CONTINUED)

- Step 6 — Add validation
- Step 7 — Verify health and API documentation
- Step 8 — Run unit tests
- Step 9 — Review logs for sensitive data

KEY TAKEAWAY Work the steps in order -- each one builds on the last.

LAB 1 — ACCOUNT SERVICE — SUCCESS CRITERIA

You are done when every box below is ticked.

SUCCESS CRITERIA

- PostgreSQL is running from docker compose
- Flyway applied V1__create_accounts
- POST /api/v1/accounts creates a PENDING account and returns 201
- GET returns the stored account
- PATCH updates nickname
- activate / freeze / close follow the status table
- Closed accounts still GET successfully (no physical delete)

SUCCESS CRITERIA (CONTINUED)

- Illegal freeze from PENDING returns 409
- Unknown account returns 404 with ACCOUNT_NOT_FOUND
- Non-synthetic customerId returns 400
- /actuator/health is UP
- Swagger UI shows the API
- mvn test passes
- Logs have no emails, card numbers, or request bodies

KEY TAKEAWAY Tick every criterion before moving on -- the next lab builds directly on this one.